# Homework 2 - AI Agents

*PromptDoctor*

Name: Mohamed Darwish Date: 7/30/2026

Github: mohddarwix/PromptDoctorx2

## Part A - Research

Cover the four questions below in your own words. Aim to explain each as if teaching a classmate who has never heard of agents.

### 1. What is an AI agent?

The best way to understand what an agent is, is by contrast. Starting with a regular script, it is a fixed-logic to follow with direct instructions. For example, read this file, extract certain information, return a result. Here, the programmer decides every step and its order, there is no intelligence where everything is given to the system and it only do instructions. There is no interactions with the outer environment.

For a single LLM prompt, the user sends a prompt to an agent that is intelligent in a sense that can produce a context output. But is still limited in the sense of text-in text-out. It can't do everything like checking the weather, saving files, or even re-planning the approach it follows to fix a failure. When a response is generated, it is done.

What makes a system a true AI agent is its ability to decide, act, observe, and fix any failure in its approach. So, an AI agent is a system that receives a certain goal, decides what actions are needed to achieve it, performs those actions, observes the results, and adjusts its next step until the goal is achieved. An agent must consist of the following: a model, a tool, memory, and control loop. The model decides what actions to do, and the surrounding program gives it access to certain tools to perform these actions, stores its observations in memory (even its failures to avoid it in the next responses), controls the repetitions, and determines when the task is done. The most important idea here is autonomy, which means that the AI can make dynamic decisions about what to do next. For example, the AI receives a goal, detects the need for a current information, calls a tool to search for this information, reads the results, notices that some data is missing, search again using different keywords, performs a calculation, and saves the result in a file. This is what differs an agent from a workflow. Instead of following fixed instructions given by a programmer, the agent decides what is necessary to do and chooses the process and tools usage.

**Comparison table:**

| Feature | Traditional script | Single model call | AI agent |
|---|---|---|---|
| **Path** | Fixed by programmer | One input-output step | Chosen dynamically |
| **Uses an AI model** | Not required | Yes | Usually |
| **Uses external tools** | Possible, but fixed | Usually no | Commonly yes |
| **Observes tool results** | Only through programmed logic | No | Yes |
| **Can change its plan** | Very limited | No | Yes |
| **Repeats actions** | Only if explicitly coded | No | Through an agent boucle |
| **Handles open-ended tasks** | Poorly | Partially | Better suited |
| **Has autonomy** | Low | Low | Moderate to high |

### 2. Types of agents

1) **ReAct (Reason and Act interactively)**

**Idea:** ReAct stands for "reasoning + acting". Instead of creating a complete plan before doing anything, the agent alternates between deciding what to do, taking actions, and observing the results. Its architecture was introduced to combine two capabilities that have been treated separately:

- Reasoning about the problem
- Interacting with the external environment via tools

**Problem solved:** LLMs that only reasons without interacting may produce outdated or incomplete answers, and LLMs that only perform actions may call tools without understanding the goal. ReAct solves this by connecting reasoning and actions into one model.

**Strengths:**

- Flexible when the environment is uncertain.
- Reacts immediately to new information.
- Can correct its direction during execution.
- Simple enough to understand and implement.

**Weaknesses:**

- Focuses mainly on the next action, so it may lose sight of the overall strategy.
- May solve complex tasks inefficiently.
- A wrong early decision can influence later steps.

**Best suited for:** ReAct is useful when:

- the next step depends heavily on the latest tool result,
- the environment changes,
- or the task cannot be fully planned.

### 2) Reflexion: Learn from Failed Attempts

**Idea:** it adds self-evaluation and memory to the agent process. After an attempt succeeds or fails, the agent examines the result and writes a short reflection about it. That reflection is stored in episodic memory and included in future attempts. Reflexion improves behaviour through feedback rather than by retraining the model or changing its weights.

**Problem solved:** A basic ReAct agent can react to observations during one execution, but it may not learn a lasting lesson from a failed attempt, which means the agent might choose always the wrong tool repeatedly. Reflexion solves this by making the agent review its performance and explicitly remember its mistakes.

**Strengths:**

- Helps the agent improve after failure.
- Makes mistakes useful.
- Does not require expensive model retraining.
- Works well when the task allows repeated attempts.

**Weaknesses:**

- The agent may produce an incorrect reflection.
- A bad lesson stored in memory can damage future attempts.
- Multiple retries increase cost and execution time.
- It requires some form of feedback or evaluation to know that an attempt failed. The agent cannot meaningfully reflect if it has no indication of whether the result was good or bad.

**Best suited for:** Reflexion is useful when:

- the task can be attempted more than once,
- feedback is available,
- tests or success criteria exist,
- and previous mistakes should influence later attempts.

### 3) Plan-and-Execute: Think Globally Before Acting

**Idea:** A Plan-and-Execute agent separates high-level planning from low-level execution. First it receives a goal, create a suitable plan, execute each step, update or revise plan if needed, and finishes. The planner considers the complete task and divides it into manageable steps. The executor then performs those steps, potentially using tools.

Problem solved: ReAct is strong at choosing the next immediate action, but it can become short-sighted. For example, ReAct agent may forget an important part of the goal. Plan-and-Execute addresses this by establishing a global structure before execution begins, and its ability to re-plan while execution.

**Strengths:**

- Keeps the complete objective visible.
- Organises long, multi-stage tasks.
- Allows different models to handle planning and execution.

**Weaknesses:**

- The initial plan may be based on incorrect assumptions.
- Planning requires extra model calls.
- Execution can become slow if every step is sequential.
- The agent may follow an outdated plan unless re-planning is allowed.
- It may over-plan simple tasks that ReAct could solve directly.

**Best suited for:** Plan-and-Execute is useful when:

- the task has clear stages,
- some steps depend on earlier steps,
- several tools must be coordinated,
- or the final result must contain multiple required sections.

#### 4) Final comparison

| Architecture | Problem it solves | Main strategy | Main weakness |
|---|---|---|---|
| **ReAct** | The agent must adapt to new tool results | Alternate decision, action, and observation | Can wander or repeat actions |
| **Reflexion** | The agent repeats mistakes across attempts | Evaluate failures and store lessons | Reflections may be wrong |
| **Plan-and-Execute** | The agent loses the overall objective | Plan globally, then execute steps | Initial plan may become outdated |

## 3. What are tools?

In the context of an AI agent, a tool is an external function that the model can choose to use when it needs to perform a specific task. For example, a tool could search the web, read a file, perform a calculation, access a database, or save a report. When the agent decides to use a tool, the model does not execute it directly. Instead, it outputs the name of the tool and the required arguments. The program running the agent, called the runtime, then executes the actual function and sends the result back to the model. The model reads this result as an observation and decides what to do next. Tools are what make agents truly useful because, without them, the model is limited to the text and knowledge already available in its context. It cannot access fresh information, read external files, save permanent results, or affect anything outside the conversation. It could explain how to send an email or calculate a result, but it could not actually perform those actions. Therefore, tools act as the connection between the model and the outside world, allowing the agent to move from simply generating text to completing real tasks.

## 4. How does an agent actually work? (The loop)

An AI agent works through a repeated loop. First, it receives an input or goal from the user. The language model then acts as the brain of the agent by understanding the request and deciding what action should happen next. It may decide to answer directly, ask for more information, or call a tool such as a web search, calculator, or file reader. The runtime executes the selected tool and returns its result to the model as an observation. The model studies this result, updates its understanding of the task, and decides whether another action is needed. This process of deciding, acting, and observing continues until the model believes it has enough information to produce the final answer.

Memory is important during this loop because it allows the agent to remember what has already happened. Short-term memory stores information from the current task, such as the original request, previous tool calls, results, calculations, and completed steps. Without it, the agent could forget what it had already done and repeat the same actions. Long-term memory stores information across different tasks, such as user preferences, previous experiences, or lessons learned from earlier mistakes. Therefore, the model makes the decisions, the tools perform external actions, and memory keeps the necessary context. The loop ends when the model decides that the goal has been completed and returns the final result to the user.

# Part B - The Project

## 5. The idea behind my agent

Most people who use AI chatbots write vague, underspecified prompts, no length, no tone, no format, no context, and get back answers that miss the mark, then blame the model instead of the prompt. Prompt engineering courses try to fix this, but they teach abstract rules and generic examples that rarely map onto the messy, specific thing you're actually trying to ask. PromptDoctor takes a different approach: instead of teaching rules, it shows you your own mistakes on your own prompts. You give it a rough prompt, it actually runs it to see what comes out, critiques the prompt itself using that output as evidence, rewrites it to fix exactly what's wrong, and repeats until the result is genuinely good then hands the improved prompt back to you. Because you watch it diagnose real weaknesses in real prompts you wrote, you're not memorizing tips, you're building the instinct for what makes a prompt work, one iteration at a time. Instead of asking an LLM to improve your prompting with an text-in text-out LLM-call, you are using an AI agent that improves your prompt using specific tools.

## 6. Design decisions

1. **Use the Claude CLI directly instead of an API key:** The agent runs on my existing Claude subscription and authenticates the same way I do.
2. **Three separate tools (executor, judge, reviser):** Collapsing "run it, critique it, fix it" into a single call makes the model grade and rewrite its own work in one call, so the model runs the prompt, tests it, fixes it, and returns the best result can be obtained.
3. **The judge critiques the prompt, but uses the prompt's actual output as evidence.** A prompt can look fine on paper and still fail in practice. Running it first and judging against real behavior catches issues a purely theoretical read would miss. Alternative: judge the prompt text alone, no execution, rejected because it's guessing, not observing.
4. **Hard iteration cap of 3, not an open-ended loop.** The model runs at most 3 iterations to get a better prompt version. Alternative: keep looping until the judge is happy, rejected because "happy" isn't guaranteed to ever happen.
5. **"Done" is decided by a single number:** judge score $\geq 8$**.**
6. **Memory is an aggregate counter of issue *types* across all past runs, not a log of past prompts.** Alternative: semantic-similarity memory over verbatim past prompts (RAG-style), rejected as unnecessary complexity for something a plain counter already solves well.
7. **Memory is saved even when the loop exits on an error:** Whatever issue types were caught before the crash are still real, useful signal.
8. **Work with all models:** The idea is to copy a prompt, fix it, and paste it back to wherever you want, so it does not interact directly with a specific LLM (like claude.ai). Alternative: use chrome extension to allow the agent to access your LLM chat directly, rejected since it adds unnecessary complexity, with no added value.
9. **Trusting the results:** Actually, there is no ground truth to decide whether a prompt is “good” or not. But since the agent is using Claude to generate responses, it is as if asking claude itself to improve this prompt but following an agentic loop not a one-call LLM.
10. **Goal is fixed:** The goal that the agent is actually working on is improving prompts.

## 7. How my agent works — walkthrough

**Step 1: You give it a rough idea.** You copy a rough prompt to your clipboard, like "write a tweet about AI", and press the hotkey. That's the entire input. Vague on purpose, because that's how people actually write prompts.

**Step 2: The agent tests it first.** Instead of guessing what's wrong with the prompt, the agent actually *runs* it through Claude to see what comes out.

**Step 3: The agent critiques the prompt, not the output.** It then asks a separate question: "Given this prompt and what it produced, what's wrong with the *prompt itself*?" It comes back with a score out of 10 and a list of specific problems. Assume it scored a low value like 2/10, flagging things like: no topic angle, no tone, no audience, no length limit.

**Step 4: It checks: good enough yet?** The rule is simple: if the score is 8 or higher, stop, it's good. 2 is far from that, so it moves on.

**Step 5: The agent rewrites the prompt.** It takes the specific list of problems and rewrites the prompt to fix exactly those, nothing more. The vague "write a tweet about AI" became something like: *"Write a tweet about how AI helps everyday workers, optimistic tone, 200–240 characters."*

**Step 6: The cycle repeats.** The agent runs the new, sharper prompt, gets a noticeably better tweet, and critiques it again.

**Step 7: It stops.** When reaching the targeted threshold, the agent declares it done and doesn't touch the prompt any further, no pointless extra polishing.

**Step 8: You get the result.** The final, improved prompt lands back on your clipboard. You paste it wherever you like and now have something specific and well-formed instead of the two words you started with.

### Tool 1: name: Executor

**Job**: runs the user's prompt and see what it produces, to get a real proof of how the prompt behave, without judging.

**Input**: current prompt text.

**Output**: claude's response on the prompt

**Why separate:** Without this step, the agent would be reasoning about a prompt in the abstract, guessing whether it's good instead of observing what it actually does. A prompt can look fine on paper and still fail in practice and the only way to catch that is to run it for real first.

### Tool 2: name: The judge

**Job:** Evaluate the prompt, not the output. It uses the output only as diagnostic evidence.

**Input:** The prompt, plus the output the Executor produced for it.

**Output:** A score out of 10, and a list of specific, named issues.

**Why separate:** Keeping judgment separate from repair keeps the critique honest, and gives the agent a clean, objective signal ("score 2/10") to decide whether to keep going.

### Tool 3: name: The reviser

**Job:** Rewrite the prompt to fix only the specific issues the Judge listed. It's not allowed to re-evaluate quality or make unrelated "improvements", it's a targeted repair step, not a second opinion.

**Input:** The current prompt, plus the Judge's list of issues.

**Output:** A revised version of the prompt.

**Why separate:** If revising and judging were the same step, the model would be grading its own rewrite. Giving the Reviser a fixed, specific list to work from also keeps its changes targeted instead of turning into a full, unrelated rewrite.

## 8. Reflection

**What was hard**

Two different kinds of hard, honestly. The first was coming up with the idea itself and being creative under time pressure. Going from "I need an agent" to a specific, defensible design took longer than writing the code. The second was making three separate tools that still had to connect into one coherent loop, deciding exactly where the Executor's job ends and the Judge's begins, and making sure each tool's output was shaped in a way the next tool could actually use.

The harder-than-expected part was getting the Claude CLI to run correctly from Python at all on Windows. Three separate bugs stacked on top of each other, one at a time: subprocess couldn't even find claude because it's an npm .cmd shim and Windows doesn't search for that the way a normal shell does; once fixed, multi-line prompts passed as command-line arguments got silently corrupted by Windows' own argument parsing, so the model kept replying "you didn't include the prompt" even though I had sent it; and then a checkmark character in a print statement crashed the whole program because the Windows console defaults to a codepage that can't display it.

**What surprised me**

How little of the difficulty was actually about the agent's reasoning, and how much was plumbing. Getting one program to correctly hand a string to another program. Once that was fixed, the "run → judge → revise" loop itself worked essentially on the first real attempt. I was also surprised by how fast convergence could be: a two-word prompt like "write a tweet about AI" went from a 2/10 to a 9/10 in a single revision cycle, not the full three iterations I expected to need. And watching the memory counter fill in over a few runs was interesting .

**What I'd improve with more time**

- **The 180-second CLI timeout is a real limitation.** It's generous for short, vague prompts, but a longer or more complex prompt could genuinely need more time, and right now that just fails outright instead of degrading gracefully.
- **A frontend would make this much more usable.** Right now the loop is invisible except for terminal text. A simple UI showing each iteration's score and issues live would let a user actually watch the reasoning happen, and step in manually if they wanted to.
- **Switching from the CLI to a direct API key would likely be faster and more robust.** Almost all the hardest bugs I hit exist specifically because I'm shelling out to another program instead of calling an API directly. An API call sidesteps all of that, and avoids the per-call CLI startup overhead too.
- **The agent could get smarter about who's using it.** Right now memory is one global counter shared by everyone. A more useful training tool would tailor its hints to the individual user's actual skill level and recurring mistakes, instead of treating a beginner and an experienced prompt writer the same way.